

SwagLabs Playwright Automation Framework

Comprehensive Technical Documentation & Repository Readme

- Playwright v1.40+
- TypeScript
- Page Object Model
- GitHub Actions CI/CD
- SauceDemo App

1. Executive Overview

This repository contains an end-to-end (E2E) automated testing suite designed for validating the functional workflows of the [SauceDemo \(Swag Labs\)](#) web application. Architected with **Playwright** and **TypeScript**, it implements the **Page Object Model (POM)** pattern alongside custom test fixtures, strong typed assertions, dynamic user data injection, and automated CI/CD pipeline integration.

Key Framework Objectives:

Maintainability via strict separation of test specs and UI locators, cross-browser support (Chromium, Firefox, WebKit), fast parallel execution, resilient auto-waiting, and clear visual trace reporting.

2. Repository Directory Structure

Below is the detailed project directory hierarchy corresponding to the codebase architecture:

```
SWAGLABS-PLAYWRIGHT/
├── .github/
│   └── workflows/
│       └── ci.yml                # GitHub Actions CI/CD workflow configuration
├── data/
│   └── users.ts                 # Test user profiles and credential mappings
├── fixtures/
│   └── test-base.ts            # Custom Playwright test fixtures for Page Objects
├── pages/
│   ├── base.page.ts            # Abstract base class with common UI helpers
│   ├── login.page.ts           # Login page locators & interaction logic
│   ├── inventory.page.ts       # Product catalog and sorting page object
│   ├── cart.page.ts            # Shopping cart verification page object
│   └── checkout.page.ts        # Checkout forms and overview page object
├── tests/
│   └── e2e/
│       ├── login.spec.ts       # Authentication & negative test suites
│       ├── inventory.spec.ts   # Catalog navigation & sorting specs
│       ├── cart.spec.ts        # Cart add/remove validation specs
│       └── checkout.spec.ts    # E2E purchase flow automation
├── utils/
│   └── constants.ts            # System messages, error strings, and URLs
├── .gitignore                  # Git exclusion configuration
├── package.json                # Project dependencies and script runner commands
├── playwright.config.ts        # Playwright global runner configuration
└── README.md                   # Project repository guide
```

3. Design Pattern & Architecture

3.1 Page Object Model (POM)

Each application web page is represented by a dedicated TypeScript class under the `pages/` directory. Pages inherit core navigation and utility capabilities from `base.page.ts`. Locators are encapsulated within page object properties, preventing selector duplication across test files.

3.2 Custom Fixtures (`test-base.ts`)

Instead of instantiating page objects manually within every test spec, custom fixtures extend Playwright's base `test` object. This allows test specs to directly receive initialized page object instances (e.g., `loginPage`) via dependency injection.

4. Test Suites & Coverage

Test Suite	File Path	Description / Scenarios Covered
Login Verification	<code>tests/e2e/login.spec.ts</code>	Successful login, locked-out user errors, invalid credentials, missing username/password field validations, and data-driven user loops.
Inventory & Catalog	<code>tests/e2e/inventory.spec.ts</code>	Product listing display, sorting by name/price (A-Z, Z-A, low-high, high-low), adding products to cart from catalog.
Shopping Cart	<code>tests/e2e/cart.spec.ts</code>	Cart badge count updates, verifying item details in cart, removing items from cart.
Checkout Workflow	<code>tests/e2e/checkout.spec.ts</code>	Complete end-to-end purchase journey, mandatory form field validation, price summary total calculation, order placement confirmation.

5. Installation & Local Setup Guide

5.1 Prerequisites

- **Node.js:** v18.0.0 or higher
- **npm:** v9.0.0 or higher
- **Git:** Installed on local system

5.2 Environment Setup

```
# Step 1: Clone the repository
git clone https://github.com/your-username/swaglabs-playwright.git

# Step 2: Navigate into the project folder
cd swaglabs-playwright

# Step 3: Install dependencies
npm install

# Step 4: Install required browser binaries with OS dependencies
npx playwright install --with-deps
```

6. Execution Commands

6.1 Running Tests

```
# Run all test suites in default headless mode
npx playwright test

# Run tests in headed (interactive UI) mode
npx playwright test --headed

# Run a specific test file
npx playwright test tests/e2e/login.spec.ts

# Run tests across targeted browsers
npx playwright test --project=chromium
npx playwright test --project=firefox
npx playwright test --project=webkit
```

6.2 Interactive Debugging & Reporting

```
# Launch Playwright Interactive UI Mode
npx playwright test --ui

# Open HTML execution report
npx playwright show-report

# Inspect visual trace files for failed test runs
npx playwright show-trace playwright-report/trace.zip
```

7. CI/CD Integration (GitHub Actions)

The framework contains an automated pipeline defined in `.github/workflows/ci.yml`. On every push or pull-request target to the `main` branch, the pipeline executes the entire E2E test suite in headless mode and uploads test reports and traces as artifacts.

```
name: Playwright E2E Tests
on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright Browsers
        run: npx playwright install --with-deps
      - name: Run Playwright tests
        run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30
```